About Software Dependency Standard - (SDS)

About Software Dependency Standard - (SDS)

Canonical Definition

The Software Dependency Standard (SDS) governs the identification,

introduction, recognition, evaluation, authorization, maintenance, change, and continuity of software dependencies upon which a software system relies for its creation, operation, modification, or continued support.

SDS establishes the methodological conditions under which a software

dependency may be recognized as a governed relationship between a software system and an internal or external software component, library, service, platform, interface, execution environment, or other software-related resource.

It governs dependency identity, origin, purpose, necessity, authority,

compatibility, constraints, criticality, risk, evidence, and lifecycle continuity.

SDS distinguishes the technical availability of a dependency from

legitimate acceptance, the ability to install a component from authority to introduce it, and current functionality from evidence that the dependency can remain suitable and supportable throughout its intended use.

Its principal output is:

Governed Software Dependency State

Its foundational distinction is:

Software Dependency Availability ≠ Governed Software Dependency

Acceptance.

• What It Is

SDS is the dependency-governance standard within SLGA™.

It establishes a technology-independent methodology for recognizing and

governing the software resources upon which applications, services, platforms, and other software systems materially depend.

SDS governs dependencies introduced through human development, AI-assisted coding, autonomous software creation, automated build processes, external suppliers, and software integration.

It addresses direct and transitive dependencies, internal components,

external libraries, runtime platforms, build environments, service interfaces, and other software-related resources whose behavior or availability materially affects the dependent software system.

SDS establishes the conditions for dependency identification, origin and

traceability, suitability, legitimate introduction, change, risk assessment, and continuity.

It treats dependency governance as an ongoing lifecycle responsibility

rather than a one-time technical installation decision.

• What It Is Not

SDS is not a package manager, dependency scanner, vulnerability database, software repository, or automated update service.

It does not independently certify that a dependency is secure, functionally correct, legally licensed, or free from defects.

It does not replace Software Security Standard (SSS), Software Quality Standard (SQS), Software Configuration Standard (SCFS), or competent legal and contractual assessments.

SDS does not require organizations to avoid open-source software, external services, cloud platforms, or third-party components.

It does not assume that commercial software is inherently more trustworthy than publicly available software.

It does not establish that a dependency is acceptable merely because it can be downloaded, installed, compiled, or executed.

SDS governs the conditions under which software reliance is legitimate and sustainable—not merely whether a component works.

• Why It Is Important

Modern software systems are built upon extensive networks of dependencies.

A single application may rely on hundreds of packages, libraries, services, runtime components, build tools, and transitive dependencies.

Many of these relationships are invisible to the end user and may not be fully understood by the organization operating the software.

A small dependency can become a critical operational point of failure.

An external service can change its interface, pricing, availability, or support conditions.

A widely used component can become unmaintained.

A transitive dependency can introduce risks that were never considered during the original development process.

Autonomous coding agents can introduce new software dependencies faster than conventional organizational review processes can recognize them.

These problems are not limited to cybersecurity.

They concern authority, reliance, compatibility, continuity,

maintainability, and the ability to reconstruct why a software system depends on particular resources.

A software system may be fully functional while remaining structurally dependent on resources that nobody has sufficiently identified, authorized, or evaluated.

SDS establishes the governance layer required to make those relationships

visible and assessable.

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

SDS occupies the third Parent Standard position:

Software Dependencies

Its principal upstream references are:

Software Requirements Standard (SRS) — Establishes the governed

requirements basis against which dependency necessity and purpose can be assessed.

Software Creation Standard (SCS) — Establishes the governed creation

context in which software components and dependency relationships are introduced.

Its principal output is:

Governed Software Dependency State

This output supports downstream configuration, quality, security, release,

deployment, change management, retirement, and lifecycle control governance.

Dependency governance continues throughout the software lifecycle and is not completed merely because the software has been created.

Standard Governance Flow

SDS contains five Core Modules.

CIM — Component Identification Module

Identifies material software dependency components, their identities,

versions, roles, and relationships to dependent software objects.

↓

DOTM — Dependency Origin & Traceability Module

Governs the origin, provenance, source relationships, and

evidence-supported traceability of direct, transitive, generated, and externally supplied software dependencies.

↓

DSM — Dependency Suitability Module

Evaluates dependency purpose, necessity, compatibility, constraints, and

suitability for the intended software context.

↓

CICM — Component Introduction & Change Module

Governs dependency introduction, inclusion, modification, replacement,

updates, and removal under applicable authority and change conditions.

↓

DCRM — Dependency Continuity & Risk Module

Governs dependency criticality, material risks, maintenance, availability,

concentration, substitutability, and continuity throughout the software lifecycle.

Together, these modules establish the SDS governance flow:

Component Identification → Dependency Origin & Traceability → Dependency

Suitability → Component Introduction & Change → Dependency Continuity & Risk

The flow represents methodological governance relationships rather than a

mandatory sequential development procedure.

Practical ApplicationsAI-Generated Software

SDS supports the recognition and governance of dependencies introduced by

AI coding assistants and autonomous development systems.

It distinguishes an agent's ability to install software from legitimate

authority to introduce new dependencies. Enterprise Software Development

Organizations can establish consistent dependency identification, suitability, approval,

Core Governance Distinctions — Continued

Package Installation Capability ≠ Dependency Introduction Authority

Technical permission to install a package does not establish legitimate authority to introduce a new dependency into a governed software system.

Dependency Utility ≠ Dependency Necessity

A component may provide useful functionality without being necessary or justified for the intended software purpose.

Package Manifest Entry ≠ Complete Dependency Recognition

A dependency manifest may omit runtime services, external interfaces, indirect reliance, or other material dependency relationships.

Dependency Name ≠ Verified Dependency Identity

A package name does not independently establish which component, version, source, or artifact is actually being used.

Claimed Dependency Origin ≠ Verified Dependency Provenance

A stated publisher or repository does not independently establish authentic origin or a sufficiently supported provenance history.

Installability ≠ Dependency Compatibility

A component may install successfully while remaining incompatible with the intended execution environment, interfaces, or software requirements.

Dependency Size ≠ Dependency Criticality

A small library or external service may represent a critical operational dependency for an entire software system.

Alternative Availability ≠ Effective Dependency Substitutability

The existence of another product does not establish that the existing dependency can be replaced without material technical or operational disruption.

Current Dependency Functionality ≠ Dependency Continuity Assurance

A dependency that functions today may become unavailable, unsupported, incompatible, or unsuitable tomorrow.

Functional Dependency ≠ Maintained Dependency

A component may continue operating after its maintainer has discontinued support and development.

Autonomous Dependency Selection ≠ Autonomous Dependency Authorization

An AI agent's technical ability to discover and select a dependency does not create legitimate authority to introduce it.

Generated Dependency Manifest ≠ Verified Dependency Reality

A generated manifest may fail to represent the complete set of resources upon which the software actually relies.

Dependency Acceptance at Creation ≠ Dependency Acceptance for Every Later

Lifecycle State

Dependency acceptance must remain subject to reassessment when software, technical, operational, or supplier conditions materially change.

Dependency Accepted ≠ Software Product Accepted

A governed dependency state does not independently establish software correctness, security, quality, or release readiness.

Use Case 1 — An AI Agent Builds Working Software by Introducing an

Unapproved Dependency

An organization authorizes an autonomous AI coding agent to develop an

internal financial reporting application.

The agent receives legitimate development requirements and access to the

software repository.

During implementation, the agent identifies an external library that can accelerate financial data processing.

The library is publicly available.

The agent installs it, integrates it into the application, and completes

the development task.

The application compiles successfully.

All initial functional tests pass.

From a conventional development perspective, the task appears complete.

However, the organization later discovers that the agent introduced an external dependency without establishing whether its inclusion was permitted.

The organization also lacks sufficient evidence concerning the dependency's

origin, maintenance conditions, and suitability for the intended financial application.

The governance problem is not that the AI agent failed to create working software.

The governance problem is that successful software creation introduced a new software reliance relationship without sufficiently establishing the authority and conditions under which that reliance was permitted.

SDS evaluates:

Dependency Identification → Origin & Traceability → Suitability →

Introduction Authority → Continuity & Risk

The assessment identifies:

• The dependency was technically available

• The agent had the capability to install it

• The resulting software was functional

• The dependency's inclusion authority remained unresolved

• Its suitability and continuity conditions had not been sufficiently

established.

The relevant dependency may receive the governance state:

Authority-Unresolved

or:

Evidence-Limited

until competent assessment establishes the applicable conditions.

The critical distinctions are:

Autonomous Dependency Selection ≠ Autonomous Dependency Authorization.

Package Installation Capability ≠ Dependency Introduction Authority.

Working Software ≠ Governed Software Dependency Acceptance.

The result: SDS exposes a governance failure that conventional build success and functional testing alone cannot resolve.

Use Case 2 — A Critical Software System Fails Because a Small

Dependency Was Never Governed as Critical

An organization operates a software platform supporting essential business

operations.

The platform has been functioning reliably for several years.

Its software contains numerous internal and external dependencies.

One relatively small external component supports a critical communication

function.

The component represents only a minor fraction of the total software

codebase.

Because of its limited size, the organization has never treated it as a

significant operational dependency.

Over time, the external maintainer discontinues support.

No replacement strategy is established.

No effective substitutability assessment is performed.

The dependency remains functional, so the organization continues using it.

Eventually, an infrastructure update makes the component incompatible with

the platform's operating environment.

The communication function fails.

The resulting disruption affects multiple dependent business processes.

The immediate technical problem is a compatibility failure.

The underlying governance problem is that a critical software reliance relationship remained operationally invisible until its failure exposed the dependency's significance.

SDS evaluates:

Dependency Identity → Criticality → Maintenance → Compatibility →

Substitutability → Continuity

The assessment identifies:

• The dependency was technically functional before the incident

• Its maintenance support had ended

• Its operational criticality had not been sufficiently recognized

• No effective replacement pathway had been established

• The software system's continuity depended on an inadequately governed

component.

The relevant dependency could have been recognized as:

Continuity-at-Risk

before the operational failure.

The critical distinctions are:

Dependency Size ≠ Dependency Criticality.

Functional Dependency ≠ Maintained Dependency.

Alternative Availability ≠ Effective Dependency Substitutability.

Current Dependency Functionality ≠ Dependency Continuity Assurance.

The result: SDS establishes why dependency governance must address criticality and continuity before technical failure occurs—not merely document the failure afterward.

Canonical Closing Statement

Modern software is built on dependencies.

Some are visible.

Others are buried inside libraries, services, platforms, build systems, and transitive dependency networks.

Some are selected by developers.

Others are introduced automatically by tools and autonomous AI agents.

Every material dependency creates a relationship of reliance.

And every relationship of reliance creates conditions that must remain governable.

A software system may work perfectly while depending on components whose origins are uncertain.

It may pass every functional test while relying on a service that can disappear tomorrow.

It may be created by an authorized AI agent while containing dependencies that the agent had no authority to introduce.

And it may appear technically stable while one unmaintained component represents a critical point of failure.

The absence of software failure is not evidence of dependency governance.

Technical availability does not establish legitimate acceptance.

Successful installation does not establish introduction authority.

Present functionality does not establish future continuity.

SDS introduces a fundamental governance requirement:

Software systems must not merely identify what they use. They must establish what they are legitimately permitted to rely upon, why that reliance is justified, and under which conditions it can continue.

This principle applies regardless of whether the dependency was introduced

by a human developer, an organization, an automated tool, or an autonomous AI agent.

The actor may change.

The technology may change.

The dependency network may grow.

The requirement for governable software reliance remains.

SDS establishes the methodological foundation for recognizing dependency

relationships as continuing governance obligations rather than isolated technical implementation choices.

A dependency is not governed because it exists, installs, or works. It is governed when its identity, purpose, authority, suitability, risks, and continuity are sufficiently established and remain subject to reassessment.

SDS transforms software dependency management into governance of software

reliance — making visible not only what a system depends on, but whether that dependence is legitimate, justified, sustainable, and sufficiently understood before its failure becomes an operational reality.